

SHAH INVESTOR'S HOME LIMITED

UAT Deployment Guide

SIHL Synapse, including the Issue Tracker & Service Request module

Document	UAT Deployment Guide
System	SIHL Synapse — one application, one database
Scope	Issue Tracker & Service Request System, BRS Phase 1 (steps 0–8)
Repository	github.com/happymanesh/synapse (private) — one repository, not two
Organisation	Shah Investor's Home Limited (SIHL), SEBI-registered stockbroker
Version	1.0
Audience	UAT deployment team, infrastructure, QA leads
Date	08-Sep-2026

Contents

1. What you are deploying
2. Environment variables
3. Deployment steps
4. First login and roles
5. Verification checklist
6. Deliberately incomplete in this release
7. Rolling forward
8. Rollback
9. Traps — read before deploying
10. Reference

1. What you are deploying

One Next.js application and one PostgreSQL database. That is the whole system.

One repository, not two. The Issue Tracker is not a separate application, repository or database — it is a module inside Synapse, surfaced as a “Support” app in the app switcher. This was a deliberate architectural decision (BRS §10): the design treats the Synapse app itself as one of its intake channels, and reads Synapse’s `user_details` master data directly to identify who raised a ticket. Splitting it out would have required an API into that data and a second authentication system. Clone one repository, provision one database, set one set of variables.

Layer	Choice
Framework	Next.js 16 (App Router, Turbopack)
Runtime	Node.js 20+ (developed on 24.15)
Package manager	npm — not pnpm, see §9
Database	PostgreSQL 16
ORM	Prisma 7 with the <code>@prisma/adapters-pg</code> driver adapter
Authentication	<code>jose</code> JWT in an httpOnly cookie, 8 hour expiry

2. Environment variables

Set these on the UAT host. `.env` is gitignored and intentionally absent from the repository — you are creating these values, not copying them from anywhere.

2.1 Required

Key	Format	Purpose
<code>DATABASE_URL</code>	<code>postgresql://user:pass@host:5432/sihl_synapse_uat</code>	Main connection. Read by the application, the Prisma CLI and the seed.
<code>SESSION_SECRET</code>	64 hexadecimal characters	Signs the session JWT. Generate a new one for UAT — do not reuse production's.

The application will not start without `SESSION_SECRET`. It is read at module load in `src/lib/session.ts` and throws if missing. That is deliberate: a missing secret fails loudly at boot rather than silently issuing tokens nobody can verify. If the app refuses to start, check this first.

Generate one with:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

2.2 Strongly recommended

Key	Value for UAT	Purpose
<code>TZ</code>	<code>Asia/Kolkata</code>	Without it the container runs UTC and every timestamp displays 5h30m behind IST — ticket raised times, the audit trail, triage-queue ages. Display only (Postgres stores UTC either way), but for a system whose purpose is measuring turnaround and holding an audit trail, wrong wall-clock times are materially misleading. This never reproduces on an IST developer machine, so it is invisible until deployed.
<code>SEED_DEMO</code>	<code>false</code>	Omits the demo fixtures — the sample accounts <code>Manesh001 / Client001</code> , the hand-created <code>sample_sales</code> table and the Sales Report worked example. Leave unset only if UAT explicitly wants that sample data.

DATABASE_URL_READONLY	connection string for a SELECT-only role	Second connection used only to execute administrator-authored report SQL. <code>assertSelectOnly()</code> already rejects non-SELECT statements, but that is an application guard; pointing report execution at a role with no write grants makes the restriction structural. Without this the app falls back to the main connection and <code>REPORT_DB_IS_READONLY</code> reports false.
-----------------------	--	---

2.3 Used by the seed only

Key	Purpose
SEED_USER_PASSWORD	Password for the account the seed creates. Falls back to the placeholder <code>ChangeMe@2026</code> if unset, so a clone of the repository can never contain a working credential. Set this to something disposable for UAT. Must satisfy the password policy: 8+ characters, at least one uppercase, one digit, one of <code>!@#\$%^&*</code> , and not equal to the username.

2.4 Set by the platform

Key	Notes
NODE_ENV	<code>production</code> . Most hosts set this. Affects Prisma client caching and the cookie <code>secure</code> flag.
PORT	Next.js binds this if present. Managed hosts inject it.

Nothing else is read. The complete list of variables the code touches is `DATABASE_URL`, `DATABASE_URL_READONLY`, `SESSION_SECRET`, `SEED_USER_PASSWORD`, `SEED_DEMO` and `NODE_ENV`, plus `TZ` which Node reads directly.

3. Deployment steps

3.1 One-time database setup

Create an empty PostgreSQL 16 database. Do not restore anything into it — the migrations build the entire schema.

Optionally create the read-only role for the report engine:

```
CREATE ROLE synapse_ro LOGIN PASSWORD '<password>';
GRANT CONNECT ON DATABASE sihl_synapse_uat TO synapse_ro;
GRANT USAGE ON SCHEMA public TO synapse_ro;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO synapse_ro;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO synapse_ro;
```

Then point `DATABASE_URL_READONLY` at it.

3.2 Install and build

Step 1 `npm ci` — runs `prisma generate` via the postinstall hook

Step 2 `npm run build`

`prisma generate` must run before `next build`. If you build in a container that prunes development dependencies, make sure it happens after install.

3.3 Apply the schema

Step 3 `npx prisma migrate deploy`

This applies the 15 committed migration files in order. See §9 — do not use `prisma migrate dev` or `prisma db push`.

3.4 Seed reference data

Step 4 `npx prisma db seed`

Creates the company, 10 hierarchies, client categories, the apps (Broking, Back Office, Support), the full menu tree, roles (`ADMIN` , `SUPPORT_DESK` , `SUPPORT_LEAD` , `PRODUCT_OPS`), role-to-menu grants, 8 applications and 7 segments, and the ticket report definitions — plus one `ADMIN` account using `SEED_USER_PASSWORD`.

The seed is idempotent (upserts throughout) and safe to re-run. It deliberately does not touch `passwordHash` on an existing account, so re-seeding never resets a password someone has changed.

3.5 Start

Step 5 `npm run start`

`start` is wired as `npm run db:deploy && next start`, so steps 3 and 4 run automatically on every start. That is intentional for a single-instance deployment and both steps are idempotent.

If UAT runs more than one instance, remove `db:deploy` from the `start` script and run it as a separate release step. Concurrent container starts would otherwise race on the same migration.

4. First login and roles

Sign in at `/login` as `Admin001`, using whatever you set `SEED_USER_PASSWORD` to. Then, immediately:

1. Change the password — user menu → Settings → Change password.
2. Create real staff accounts in Administration → User Master.
3. Assign roles in Administration → Roles, or via the shield action on a user.

Role	Can
<code>SUPPORT_DESK</code>	Triage, resolve, forward and merge tickets; log tickets for others; propose change requests
<code>SUPPORT_LEAD</code>	All of the above, plus maintain the Applications and Segments master lists
<code>PRODUCT_OPS</code>	Approve or reject change requests and manage the backlog
<code>ADMIN</code>	Everything, including all Administration screens

The split between proposing and approving a change request is a stated requirement — BRS §4.6: the support desk cannot unilaterally park work in the backlog — and is enforced server-side by a role check, not by hiding a button.

Menu grants are filtered by the user's own hierarchy. A role granted menus at hierarchy `0200` shows nothing to a user at `0100`. The seed grants the support roles across `0000/0100/0200/0300`, so assigning a support role to any staff user works — but if you add roles by hand, grant them at the hierarchies your users actually hold, or they will sign in to an empty app.

5. Verification checklist

#	Check	Expected
1	/login	200, SIHL logo and sign-in form render
2	Unauthenticated page, e.g. /support/queue	Redirects to /login
3	Unauthenticated API, e.g. /api/notifications	401 with a JSON body — not an HTML redirect
4	Sign in	Dashboard, with Support in the app switcher
5	Raise a Ticket	Three dropdowns: Type, Application, Segment
6	"Raising at" line	Shows IST , not UTC — this is the TZ check
7	Select type "Others"	Reveals the "Please mention the type" field
8	Select "+ Add a new application..."	Reveals a name field
9	Raise a ticket	Appears in My Tickets and Triage Queue
10	Ticket detail page	History panel shows a CREATED event
11	Daily Ticket Stats, Tickets by Application	Run, and export CSV

6. Deliberately incomplete in this release

These are known and agreed, not defects. Brief UAT testers so they are not raised as bugs.

Area	State
Outbound delivery	Nothing is ever sent. Acknowledgements, change-request closure notices and document deliveries are written to the <code>channel_message</code> table with status <code>QUEUED</code> . Synapse has no email or SMS provider wired, so a raiser will hear nothing.
Document generation (§4.7)	Stubbed. A request is recorded and its delivery address resolved and verified, but no PDF is produced — the ledger, margin-report and contract-note source tables do not exist and no PDF renderer is installed.
Website intake (§4.1)	Not built. There is no public, unauthenticated intake route. Only the Synapse app and staff-logged Phone/WhatsApp entries create tickets.
WhatsApp intake (§4.1)	Parked, pending a centralised WhatsApp system. Requires Meta credentials and a public HTTPS callback.
Email intake, CEO dashboard, solutions bot	Phase 2 and Phase 3 by design (BRS §7).
Issue taxonomy	The 8 applications and 7 segments seeded are provisional and were not signed off by the support lead (BRS §11.2). Both are editable in-app under Support → Applications / Segments.
Daily morning reminder (§4.5)	Surfaces as counters on the Triage Queue rather than a pushed reminder, because pushing depends on delivery being wired.

7. Rolling forward

For future releases, pull and then:

```
npm ci && npm run build && npx prisma migrate deploy && npx prisma db seed
```

Schema changes in this project use hand-written migration SQL committed under `prisma/migrations/`. `migrate deploy` applies them in order and never inspects the live schema, so it is safe. See §9.

8. Rollback

The application is stateless — redeploy the previous commit.

The database is not. Migrations here have no down-scripts, so take a database snapshot before applying any release that includes new migrations. The practical rollback is: restore the snapshot, then redeploy the matching application commit.

9. Traps — read before deploying

Each of these cost real debugging time. None are guessable from the code.

1. Never run `prisma migrate dev` or `prisma db push`. The database contains `sample_sales`, a hand-created table that is not a Prisma model. Both commands plan to **drop it** to reconcile drift. Only `prisma migrate deploy` is safe: it applies committed migration files in order and never diffs the live schema.

2. A page that queries the database must not be statically prerendered. `next build` will try to reach the database at build time, which fails in any environment whose build step cannot see it. Every route is already dynamic (`/login` carries an explicit `export const dynamic = "force-dynamic"`), but keep it that way when adding pages. This passes locally, where build and app share one database, and fails only in CI or a hosted builder.

3. `TZ` must be set, or timestamps are 5h30m out. Invisible on an IST developer machine. See §2.2.

4. A raw `node-postgres` script disagrees with the app about timestamps. `pg` parses `timestamp without time zone` as *local* time; Prisma reads it as UTC. The same row therefore reads 5h30m apart depending on which client you use. The app is correct — do not “fix” a timestamp based on an ad-hoc `psql` or `pg` probe.

5. Menu grants are filtered by the user’s hierarchy. See §4.

6. Use `npm`, not `pnpm` — `pnpm` had approve-builds problems on the original machine.

7. After `prisma generate`, restart the server. A running process keeps the old Prisma client in memory, so new models appear “missing” at runtime while the code type-checks fine.

10. Reference

Document	Contents
REBUILD.md	Full product brief — data model, architecture rules, feature spec, traps
CLAUDE.md	Day-to-day conventions: migrations, report SQL, ownership columns, formatting
docs/01-business-requirements	Synapse platform BRS
docs/02-technical-architecture	Architecture document
docs/03-user-guidelines	User guide
docs/04-issue-tracker-brs	Issue Tracker BRS — the specification this module implements
docs/05-uat-deployment	This document

Run `npm test` for the unit suite — 223 tests, no test-framework dependency (`node:test` via `tsx`).